

# 第 42 讲: Train / Validation / Test

为什么三个数据集必须严格分工, 以及 Data Leakage 到底有多危险

《面向 LLM / Agent Systems 科研的深度学习零基础课》

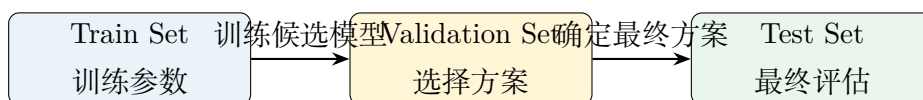
## 【科研洞察】

这一讲不是“数据集怎么随便分成三份”的操作课, 而是在建立一条科研底线: **评估数据一旦影响了模型选择, 它就不再是独立评估数据**。以后你做 LLM、Agent Systems、微调、Benchmark 或 Ablation, 只要这条边界被破坏, 分数就可能比真实泛化能力更漂亮。

## 一、这一讲要解决什么问题?

上一讲已经看到: Training Loss 很低, 并不保证 Generalization 好。那下一步自然出现一个问题: **我们怎样知道模型面对“没参与训练的新数据”到底行不行?**

最基本的答案是把数据分工:



但是, 真正困难的不是会不会写 `train_test_split`, 而是回答:

- 什么才算“独立样本”? 一行数据? 一个用户? 一个病人? 一段 trajectory? 一个任务模板?
- Validation 能看多少次? 反复调参会不会把 Validation 也“调过拟合”?
- Test 可不可以看一眼再改模型?
- 标准化、特征选择、Tokenizer、检索库、数据去重应该在 split 前还是 split 后做?
- LLM 的 benchmark contamination、Agent trajectory leakage 与普通表格数据泄漏是什么关系?

## 二、Train / Validation / Test 各自到底做什么?

### 1. Train Set: 允许直接改变 Parameter

#### 【原理】

Train Set 的职责是提供训练信号。Forward 得到预测, Loss 衡量错误, Backward 计算 Gradient, Optimizer 更新 Parameter。Train 数据可以直接影响 Weight / Bias 等可训练参数。

例如:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} L_{train}(\theta_t)$$

这里的  $L_{train}$  就来自 Train Set。

2. Validation Set：不能直接训练 Weight，但会影响“我们选什么”

Validation Set 常用于：

- 选择 Learning Rate、Batch Size、Weight Decay；
- 选择网络宽度、深度、Dropout 比例；
- 选择模型结构或 Prompt / Agent policy 方案；
- Early Stopping；
- 选择哪个 checkpoint 作为最终模型；
- Ablation 后决定保留哪个组件。

它虽然通常不直接进入 `loss.backward()`，但会影响我们的决策。因此从统计意义上说，Validation 也参与了“模型开发”。

3. Test Set：只回答最后一个问题

Test Set 的核心职责是：

当模型、超参数、checkpoint、评估协议都定下来以后，它在真正未参与开发的数据上表现如何？

【常见误区 / 风险】

如果你看了 Test 结果以后说“这个结构不行，我换一个再测”，那么 Test 已经开始承担 Validation 的职责。重复几次以后，这个 Test 分数就不再是严格独立的最终估计。

三、Parameter 与 Hyperparameter：为什么 Validation 也会影响模型？

| 项目                | Parameter                | Hyperparameter / 设计选择                          |
|-------------------|--------------------------|------------------------------------------------|
| 典型例子              | Weight、Bias、Embedding 参数 | LR、Batch Size、层数、Hidden Size、Dropout、Prompt 模板 |
| 谁来更新              | Backprop + Optimizer     | 人工选择、搜索算法、Validation 比较                        |
| 直接使用 Train        | 是                        | 通常间接使用                                         |
| 是否受 Validation 影响 | 通常不直接                    | 是                                              |

Validation 虽然不直接改 Weight，但它会让我们决定“下一次训练用什么配置”。从更高层看，这也是一种反馈回路：

Validation Result → Researcher Decision → New Model

所以 Validation 不是无限可用的“免费数据”。

## 四、一个完整数字例子：1000 条数据怎样分工？

假设有 1000 条独立样本，采用：

70%/15%/15%

则可以得到：

700 Train, 150 Validation, 150 Test

流程应当是：

1. 用 700 条 Train 训练若干候选模型；
2. 在 150 条 Validation 上比较候选方案；
3. 确定 architecture、hyperparameters、checkpoint；
4. **冻结开发决策**；
5. 最后一次在 150 条 Test 上报告最终结果。

### 【具体例子】

本讲实验 1 使用 600 个样本，按 360 / 120 / 120 切分。候选多项式 degree 为 1、3、5、7、9、12、15，只按 Validation MSE 选择。最终 Validation 选择 degree=9，对应 Test MSE=0.038365。

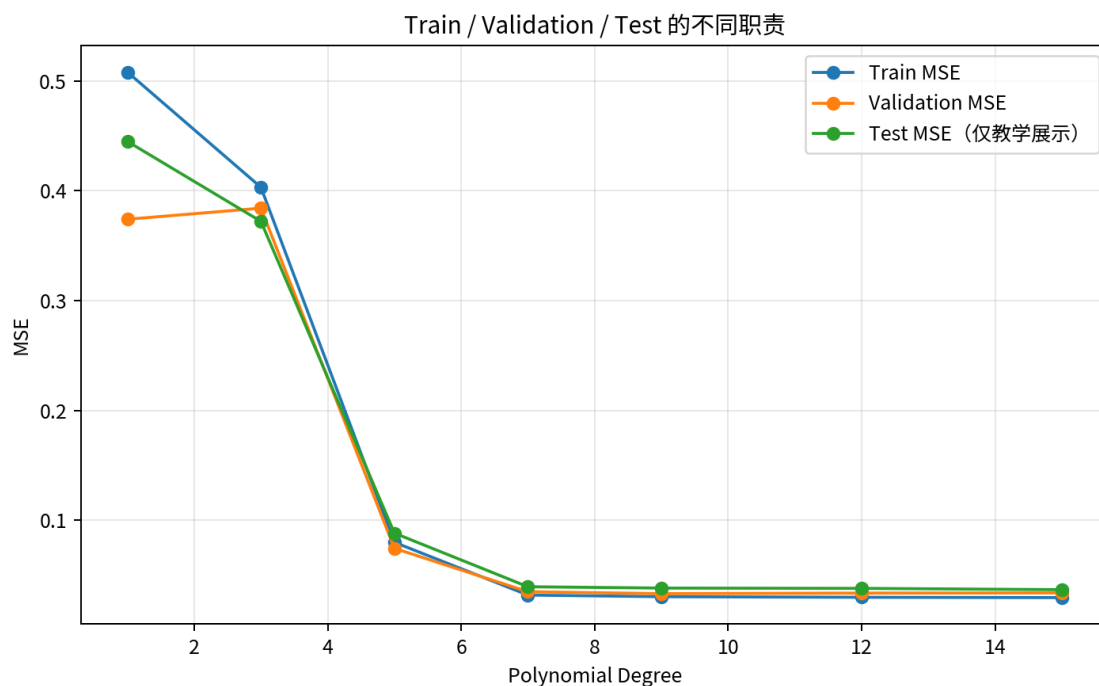


图 1: Train / Validation / Test 的职责不同。教学图中展示 Test 曲线，但实际选模型时不能用 Test 曲线。

## 五、为什么不能反复看 Test？——“看答案再改题”

最通俗的类比：

- Train = 平时练习题；
- Validation = 模拟考试；
- Test = 最终考试。

如果每次最终考试后都允许回去改复习策略，然后继续参加“同一份最终考试”，你最终会越来越适应这份试卷，而不是越来越适应真实未知试卷。

数学上可以把“模型选择”也看作一个优化过程。即使每个候选模型都没有真实能力，只要候选够多，我们也能靠随机波动挑出一个在 Validation 上看起来特别好看的。

## 六、Validation 也会被过拟合：不是只有 Train 才会 Overfit

本讲实验 2 故意制造一个极端场景：标签完全随机，每个候选模型也只是随机猜测，所以真实泛化能力就是约 50%。

结果：

| 候选模型数 | 最佳 Validation Accuracy | 被选模型 Test Accuracy |
|-------|------------------------|--------------------|
| 1     | 0.4968                 | 0.4994             |
| 10    | 0.5574                 | 0.5007             |
| 100   | 0.5867                 | 0.4994             |
| 1000  | 0.6138                 | 0.4996             |
| 5000  | 0.6277                 | 0.5014             |

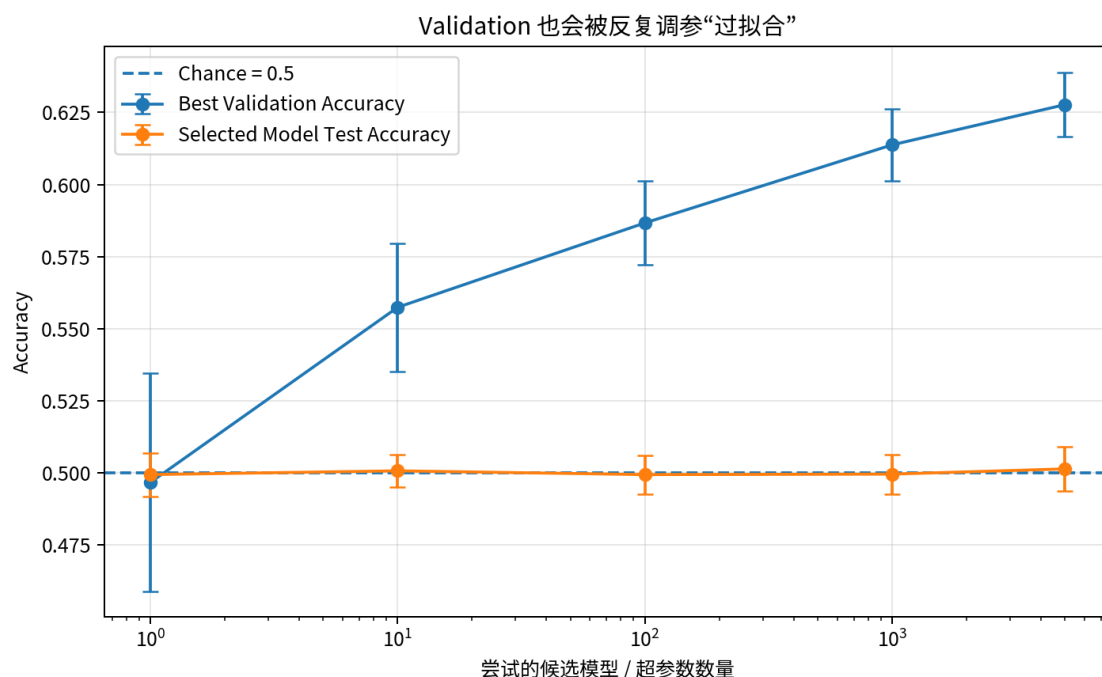


图 2: 候选方案越多, Validation 最优值越容易被随机波动“挑漂亮”, 但独立 Test 仍停留在随机水平。

**【科研洞察】**

这就是为什么大规模 Hyperparameter Search、Prompt Search、Agent strategy search 做得越多，越需要严格独立的最终评估集。**Validation** 是模型开发资源，不是无限次使用仍保持独立的真空。

## 七、Data Leakage 到底是什么？

**【原理】**

Data Leakage (数据泄漏) 可以理解为：**训练或开发过程获得了在真实部署时本不应该知道的信息，或者 Train 与 Evaluation 之间并不真正独立**。结果通常是评估指标虚高。

泄漏不一定是“直接把 Test 标签喂给模型”这么明显。更多时候它非常隐蔽。

### 类型 1: Target Leakage (目标泄漏)

例如预测“病人未来是否复发”，Feature 却包含“复发后才会生成的治疗记录”；预测 Agent 是否成功，却把任务结束后的成功标志编码进输入。

此时模型可能表现惊人，但它使用的是未来部署时拿不到的信息。

### 类型 2: Preprocessing Leakage (预处理泄漏)

错误流程：

全数据 → 计算均值/标准差 → 再 split

正确流程：

先 split → 只用 Train fit preprocessing → 对 Val/Test transform

不仅 Standardization 如此，以下步骤也可能泄漏：

- PCA / Feature Selection;
- 缺失值填补规则;
- Vocabulary 建立;
- Target Encoding;
- 数据驱动的阈值选择;
- 从全数据统计出来的 Normalization 常数。

### 类型 3: Duplicate / Near-Duplicate Leakage

如果同一原始样本的复制、切片、增强版本或近重复版本分别进入 Train 和 Test，模型可能只是认出了“同一个东西”。

LLM 中尤其要注意：Benchmark Question 的近似改写、答案解释、网络转载、同源题库都可能构成 contamination。

## 类型 4: Entity / Group Leakage

这对你的 Agent Systems 科研尤其重要。

如果同一个用户、病人、设备、文档、任务实例、Agent trajectory 被切成多条 row，然后随机按 row split，就可能出现：

同一实体的一部分在 Train，另一部分在 Test

模型不是泛化到“新实体”，而是在识别它已经见过的实体特征。

## 八、实验 3：为什么同一 Agent trajectory 不能随便按行随机切？

本讲构造 200 个 group，每个 group 有 6 条高度相似记录，并为每个 group 随机分配一个二分类标签。由于标签是随机的，面对真正 unseen group 时理论上只能接近 50%。

错误的 row-level split 得到：

$$Accuracy = 1.0000$$

因为 170 个 Test groups 全部在 Train 中也出现过。

而正确的 group-level split 得到：

$$Accuracy = 0.4472$$

接近随机猜测。

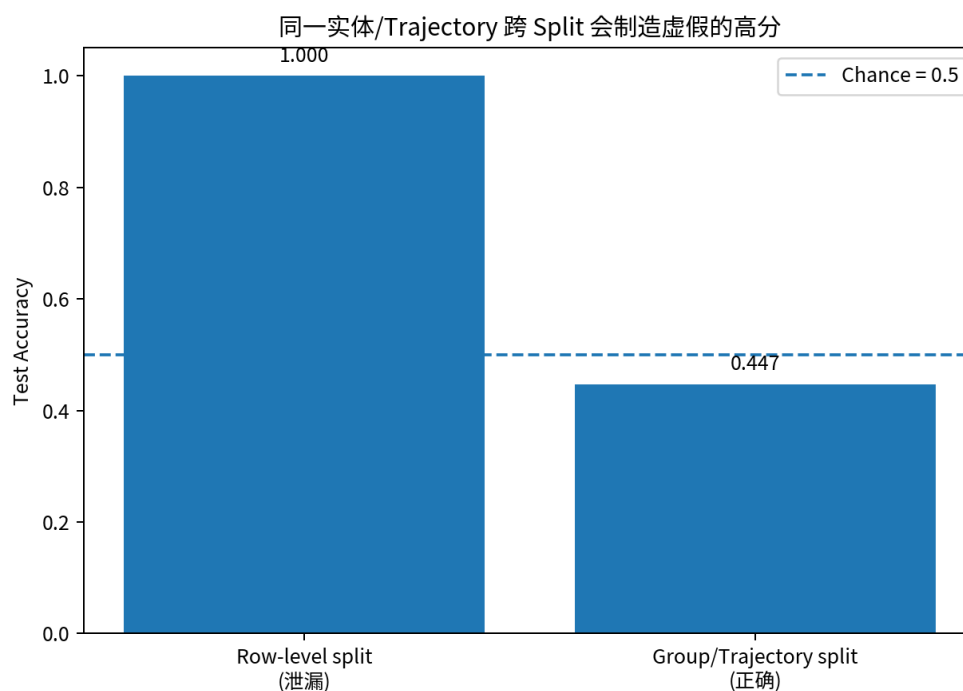


图 3: 错误的 row split 可以把“记住 group identity”伪装成“泛化能力”。

【科研洞察】

对于 Agent trajectory, 真正的独立单位往往不是 timestep, 也不是 message, 而可能是完整 episode / task instance / environment seed / user / scenario family。你要先定义“未来希望泛化到什么”, 再决定 split unit。

九、Time Leakage: 时间序列和在线系统为什么不能随便 shuffle ?

假设目标是“用过去预测未来”。如果随机 shuffle, 未来时期的样本可能进入 Train, 而较早时期的样本进入 Test。这样训练过程间接看到了未来分布。

典型正确方式:

过去 Train → 较近未来 Validation → 更后未来 Test

在 Agent / 在线系统中也类似: 如果你研究系统随版本、记忆、用户反馈逐渐演化, 不能让后期状态的信息泄漏到早期训练阶段。

十、Data Leakage 与 Distribution Shift 不是一回事

|        | Data Leakage                    | Distribution Shift                      |
|--------|---------------------------------|-----------------------------------------|
| 核心问题   | 不该知道的信息进入训练/开发                  | Train 与未来环境本来就不同                        |
| 通常影响   | 指标虚高                            | 真实部署性能下降                                |
| 解决思路   | 重做 split / pipeline / isolation | 设计更真实的 evaluation、robustness、adaptation |
| 能否同时存在 | 可以, 而且现实研究里经常同时存在               |                                         |

十一、Stratified / Group / Time Split: 到底应该选哪一种 ?

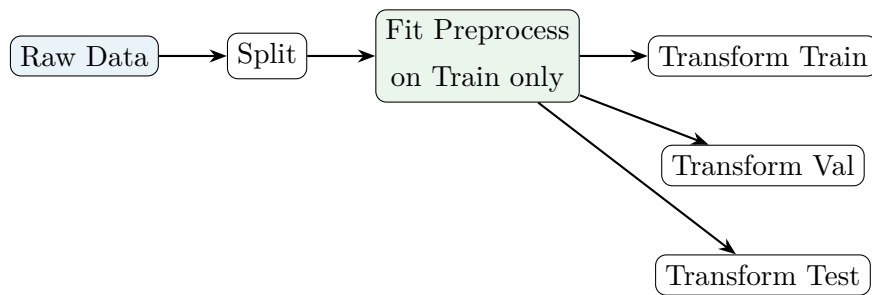
- Random Split: 样本近似独立同分布时的基础方案;
- Stratified Split: 分类任务希望各 split 类别比例相近;
- Group Split: 同一实体产生多条记录, 必须整个实体一起分配;
- Time Split: 任务具有时间方向, 部署目标是未来;
- Domain Split: 希望测试跨任务、跨环境、跨机构泛化时, 按 domain 分开。

【常见误区 / 风险】

“随机切 8:1:1” 不是默认科学答案。Split Strategy 本身必须和你的研究问题匹配。

十二、一个正确 Preprocessing Pipeline 应该怎样写 ?

正确结构:



例如 StandardScaler:

```

1 # 只在 Train 上估计 mean / std
2 mean = x_train.mean(axis=0)
3 std = x_train.std(axis=0)
4
5 x_train = (x_train - mean) / std
6 x_val    = (x_val    - mean) / std
7 x_test   = (x_test   - mean) / std
  
```

### 十三、LLM 里的 Data Leakage / Contamination

在 LLM 研究中，泄漏不一定来自你手上的 fine-tuning Dataset，还可能来自更早的 pretraining data。

需要区分：

- Benchmark 原题是否出现在预训练语料；
- 是否出现了答案、解析、转载、翻译、近重复改写；
- Instruction Tuning 或 RAG Index 是否包含评测答案；
- Few-shot Examples 是否与 Test Question 高度同源；
- Prompt 设计是否是在不断看 Test 结果后调出来的。

这也是为什么 LLM Benchmark 高分必须结合 contamination 风险解释，而不是只看一个 Accuracy 数字。

### 十四、Agent Systems 里的泄漏比表格数据更隐蔽

Agent 数据经常具有层级结构：

Environment → Task → Trajectory → Step / Message / Tool Call

如果研究问题是“能否泛化到新任务实例”，那么 split 应该至少保证 Task Instance 不跨集合；如果研究“能否泛化到新环境”，那么 Environment 都应该隔离。

常见风险：

- 同一 trajectory 的前半段在 Train、后半段在 Test；



- 同一个 task seed 的不同 rollout 跨 split;
- 测试任务的 reference answer 进入 RAG / Memory;
- 自动生成训练数据时, 把 benchmark 测试题作为 seed;
- Agent 的长期 Memory 在评估前没有 reset, 带入过去测试信息。

## 十五、Validation、Early Stopping 与 Checkpoint Selection

常见流程: 每个 Epoch 后计算 Validation Loss, 并保存最好 checkpoint。

```
1 best_val = float("inf")
2 for epoch in range(num_epochs):
3     train_one_epoch(...)
4     val_loss = evaluate(model, val_loader)
5
6     if val_loss < best_val:
7         best_val = val_loss
8         save_checkpoint(model, optimizer, ...)
```

这里要理解: 最终模型不是“最后一个 Epoch”, 而可能是 Validation 最优的那个 checkpoint。这个选择已经使用 Validation 信息, 所以 Test 必须继续保持独立。

## 十六、Cross-Validation: 数据很少时怎么办?【理解即可】

当数据量很小, 可以使用 K-Fold Cross-Validation: 把训练开发数据划成 K 份, 轮流用一份做 Validation, 其余做 Train, 再平均结果。

但仍然要保留一个思想: 用于最终报告的 Test / External Test 如果存在, 就不要参与 Cross-Validation 的选择。

## 十七、可复现性: Split 本身也必须可复现

科研代码建议保存:

- Random Seed;
- Train / Val / Test 的样本 ID 列表;
- Group / Time split 的规则;
- Dataset Version / Hash;
- 去重规则;
- Preprocessing fit 的范围;
- 是否使用外部数据、RAG、Memory;

- Benchmark contamination 检查方法。

只写“随机划分 80/10/10”通常不足以完整复现实验。

## 十八、科研论文里怎样报告才算清楚？

建议至少回答：

1. 训练、验证、测试各有多少样本 / trajectory / task?
2. 独立单位是什么？row、user、trajectory、task 还是 environment?
3. 是否 stratified / grouped / temporal?
4. Hyperparameter 根据哪个集合选择?
5. Checkpoint 根据什么标准选择?
6. Test 被评估了多少次，是否参与开发?
7. Preprocessing / Retrieval Index / Memory 是否只使用允许的数据?
8. 是否有 duplicate / near-duplicate / contamination 检查?

## 十九、常见误区

### 【常见误区 / 风险】

**误区 1：**“只要代码里没把 Test 放进 backward，就没有泄漏。”  
错误。只要 Test 结果影响了设计决策，它就已经参与开发。

### 【常见误区 / 风险】

**误区 2：**“数据很多，所以随便随机切行也没事。”  
错误。是否泄漏取决于独立单位，而不是总样本量。

### 【常见误区 / 风险】

**误区 3：**“Validation 可以无限次看，因为它不是 Test。”  
错误。反复搜索会把选择过程逐渐适配 Validation 的随机噪声。

### 【常见误区 / 风险】

**误区 4：**“Test 分数高，就说明真实部署一定好。”  
错误。即使没有 Leakage，还可能有 Distribution Shift、Benchmark 不代表真实场景等问题。

二十、与 LLM / Transformer / Agent Systems 的关系

你以前在 LLM 课程里已经见过 Train / Validation / Test，但现在要从“科研底层”重新理解：它们不是三个文件夹，而是三种**信息权限**。

| 场景               |  | 真正要问的 split 问题                      |
|------------------|--|-------------------------------------|
| LLM Fine-tuning  |  | 同一文档/同源样本是否跨 split?                 |
| Benchmark        |  | 测试题是否被训练、检索、Prompt Search 间接看过?     |
| Multi-Agent      |  | 同一 task / seed / environment 是否跨集合? |
| Long-term Memory |  | Test 前是否 reset，是否携带过去测试信息?          |
| Agent Trajectory |  | 同一 trajectory 的 step 是否被错误拆散?       |
| Scientific Agent |  | 同一论文、同一问题族、同一数据库来源是否跨 split?        |

二十一、最低闭环能力：以后拿到 Dataset 先问这 7 个问题

1. 我真正希望模型泛化到什么对象?

2. 什么才是独立单位?

3. Train / Val / Test 是按什么单位 split 的?

4. Preprocessing 有没有在 split 之前 fit?

5. 有没有 duplicate / near-duplicate / group overlap?

6. Validation 被用于多少轮选择?

7. Test 是否真的在最终方案确定之前从未参与决策?

【本节小结】

一句话总结：Train 用于学习 Parameter；Validation 用于做开发决策；Test 用于最终独立评估。Data Leakage 的本质，是本应隔离的信息跨越了这条边界。对于 LLM / Agent Systems，真正困难的往往不是“分三份”，而是定义正确的独立单位、时间边界、数据来源和信息权限。